

Solve the following problems using Spark/Scala. Do not create any map data structures. Try to avoid using traditional loops when possible.

Consider three files. One file has information about students (ID, name, address, phone number).

1, John, 123 Main, 233 223 5566
is an example.

Consider a second files that has information about courses and their difficulty.

CSC365, 1
CSC369, 1
CSC469, 2
is an example.

Consider a third file that contains the student ID, course, and grade.

For example:

1, CSC365, A
1, CSC369, A
1, CSC469, B

It contains information about student taking a class and earning a grade.

Q1[10] Write a program that finds the names of the students that have taken at least one of the courses with the greatest difficulty. In our example, John has taken such a course.

```
// Parse the input files
val studentsRdd = sc.textFile("input/asgn06/students/")
  .map(l => l.split(",")) // Comma delimited
  .map({
    // trim the result and turn into a tuple
    case Array(sid, sname, saddress, sphone) => (sid.trim.toInt, sname.trim)
    case _ => throw new IllegalArgumentException("You are the problem... You should never
be here!")
  })

val coursesRdd = sc.textFile("input/asgn06/courses/")
  .map(l => l.split(",")) // Comma delimited
  .map({
    // trim the result and turn into a tuple
    case Array(cname, cdifficulty) => (cname.trim, cdifficulty.trim.toInt)
    case _ => throw new IllegalArgumentException("You are the problem... You should never
be here!")
  })

val takenRdd = sc.textFile("input/asgn06/taken/")
  .map(l => l.split(",")) // Comma delimited
  .map({
    // trim the result and turn into a tuple
    case Array(sid, cname, grade) => (cname.trim, sid.trim.toInt)
    case _ => throw new IllegalArgumentException("You are the problem... You should never
be here!")
  })
```

```

    })

    // Most difficult course value
    val mostDifficultArray = coursesRdd
      .sortBy(t => (-t._2, t._1)) // Sort starting with the most difficult
      .take(1) // Take the first element

    val mostDifficultValue = mostDifficultArray(0)._2

    // Find the course tuples with that highest difficulty
    val difficultCourses = coursesRdd
      .filter(_._2 == mostDifficultValue)

    // Get all student ids that have at least one of the most difficult courses
    // Join those against the students to get the names
    // Printing each of the names
    difficultCourses
      .join(takenRdd)
      .map({
        case (cname, (difficulty, sid)) => (sid, ())
      })
      .distinct
      .join(studentsRdd) // (sid, ((), sname))
      .collect()
      .foreach({
        case (sid, ((), sname)) => println(sname)
      })
  }
}

```

Q2[10] Write a program that prints the average course difficulty of the classes that are taken by each student. For our example, the program will print **John, 1.33**. Make sure to print average course difficulty of 0 if the student doesn't take any classes (hint, use left outer or right outer join).

```

// Parse the input files
val studentsRdd = sc.textFile("input/asn06/students/")
  .map(l => l.split(",")) // Comma delimited
  .map({
    // trim the result and turn into a tuple
    case Array(sid, sname, saddress, sphone) => (sid.trim.toInt, sname.trim)
    case _ => throw new IllegalArgumentException("You are the problem... You should never
be here!")
  })

val coursesRdd = sc.textFile("input/asn06/courses/")
  .map(l => l.split(",")) // Comma delimited
  .map({
    // trim the result and turn into a tuple
    case Array(cname, cdifficulty) => (cname.trim, cdifficulty.trim.toInt)
    case _ => throw new IllegalArgumentException("You are the problem... You should never
be here!")
  })

val takenRdd = sc.textFile("input/asn06/taken/")

```

```

.map(l => l.split(",")) // Comma delimited
.map({ // trim the result and turn into a tuple
  case Array(sid, cname, grade) => (cname.trim, sid.trim.toInt)
  case _ => throw new IllegalArgumentException("You are the problem... You should never
be here!")
})

// Join so the difficulty is included in the courses taken
val job1 = takenRdd
  .join(coursesRdd)
  .map({
    case (cname, (sid, cdifficulty)) => (sid, cdifficulty)
  })

// Left join with students (ensures all students are covered)
// Filter out all the Nulls to be 0 as the average difficulty
val allStudents = studentsRdd
  .leftOuterJoin(job1)
  .map({
    // check for the None from the partial join
    case (sid, (sname, None)) => ((sid, sname), (0.0, 1))
    // convert the original Some() to just a double
    case (sid, (sname, Some(difficulty))) => ((sid, sname), (difficulty.toDouble, 1))
  })

// Group students by their key sid
// Get the average by aggregating against all elements
val studentAverageDifficulties = allStudents
  .reduceByKey((x, y) => (x._1+y._1, x._2+y._2)) // keep track of sum and count
  .mapValues({ case(x, y) => x*1.0/y }) // divide the sum and the count

// Print everything
studentAverageDifficulties.collect().foreach(println)

```

Q3[10]. Write a program that prints the top five most difficult classes.

```

// Parse the input files
val coursesRdd = sc.textFile("input/asgn06/courses/")
  .map(l => l.split(",")) // Comma delimited
  .map({ // trim the result and turn into a tuple
    case Array(cname, cdifficulty) => (cname.trim, cdifficulty.trim.toInt)
    case _ => throw new IllegalArgumentException("You are the problem... You should never
be here!")
  })

// Sort by difficulty descending
// Take the first 5 from that list (highest difficulty)
// Only print the name of the course
coursesRdd
  .sortBy(t => (-t._2, t._1))

```

```
.take(5)
.foreach({ case (cname, cdifficulty) => println(cname) })
```

Q4[10]. Write a program that prints the name of the students ordered by GPA in descending order (starting with the student with the highest GPA). The GPA of a student that has taken no courses should be 0 (use left outer or right outer join).

```
// Parse the input files
val studentsRdd = sc.textFile("input/asgn06/students/")
  .map(l => l.split(",")) // Comma delimited
  .map({ // trim the result and turn into a tuple
    case Array(sid, sname, saddress, sphone) => (sid.trim.toInt, sname.trim)
    case _ => throw new IllegalArgumentException("You are the problem... You should never
be here!")
  })

val takenRdd = sc.textFile("input/asgn06/taken/")
  .map(l => l.split(",")) // Comma delimited
  .map({ // trim the result and turn into a tuple
    case Array(sid, cname, grade) => (sid.trim.toInt, lToNGrade(grade.trim), 1.0)
    case _ => throw new IllegalArgumentException("You are the problem... You should never
be here!")
  })

// Get average gpa for students who have taken courses
val averageGrade = takenRdd
  .reduceByKey((x, y) => (x._1+y._1, x._2+y._2))
  .mapValues({ case (gr, ct) => gr/ct})

// Left join, turning None into 0.0 average gpa
val allGpas = studentsRdd
  .leftOuterJoin(averageGrade)
  .map({
    case (sid, (sname, None)) => (sname, 0.0)
    case (sid, (sname, Some(average))) => (sname, average.toDouble)
  })

// Sort by the gpa descending, then sname ascending
val sortedGpas = allGpas
  .sortBy({ case (sname, average) => (-average, sname) })

// Print everything
sortedGpas.collect().foreach(println)

def lToNGrade(l: String): Double = {
  return l.toUpperCase match {
    case "A" => 4
    case "B" => 3
    case "C" => 2
    case "D" => 1
    case "F" => 0
    case _ => Double.NaN
  }
}
```

$\frac{3}{5}$ $\frac{3}{5}$
